

GenAI Analysis Report

1. Tool and Access Methods:

- a) GenAI Tool 1: ChatGPT (GPT-4o) accessed on web browser at chatgpt.com
- b) GenAI Tool 2: Claude (Claude 3.5 Sonnet) accessed on web browser at claude.ai

2. Prompt Used:

“Write a complete, single-file C++ object-oriented program that prioritizes emails for a company CEO using a custom list-based MaxHeap priority queue based on the following specification:

Implementation Rules:

1. Use pure C++ Object-Oriented Design (classes and objects). Do not use procedural or standalone functions for core data structure operations.
2. Implement the MaxHeap priority queue entirely from scratch using a list-based (dynamic array or vector) structure.
3. DO NOT include or use `<queue>`, `std::priority_queue`, or standard algorithm heap functions (such as `std::push_heap` or `std::pop_heap`).
4. Read commands line-by-line from standard input (`std::cin`) until end-of-file (EOF).

Email Priority Rules:

- Sender priority hierarchy (highest to lowest): Boss > Subordinate > Peer > ImportantPerson > OtherPerson.
- Tie-breaking: If sender categories match, prioritize the newest date (MM-DD-YYYY).

Commands to Handle:

- EMAIL `<sender category>`, `<subject line>`, `<date>`: Inserts email into MaxHeap.
- NEXT: Displays the next highest-priority email without removing it. Format:
Next email:
Sender: `<sender>`
Subject: `<subject>`
Date: `<date>`
- READ: Removes the highest-priority email from the heap without printing.
- COUNT: Displays current unread count as "There are X emails to read."

Edge Cases:

- Handle NEXT or READ when queue is empty (do not crash).
- Multiple consecutive NEXT commands should show the same top email.
- Multiple consecutive READ commands quietly remove emails one by one.

Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT

Expected Output:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Provide a complete, fully functioning C++ file including main().”

3. Raw Generated Code

NOTE: The code is split up into pieces through screenshots, as the code is too long for one photo.

ChatGPT:

```

1
2 #include <iostream>
3 #include <vector>
4 #include <string>
5 #include <sstream>
6 using namespace std;
7
8 class Email {
9 private:
10     string sender;
11     string subject;
12     string date;
13
14     int senderPriority;
15     int dateValue;
16
17 public:
18     Email(string s, string sub, string d) {
19         sender = s;
20         subject = sub;
21         date = d;
22
23         // Assign priority based on sender category
24         if (sender == "Boss") {
25             senderPriority = 5;
26         }
27         else if (sender == "Subordinate") {
28             senderPriority = 4;
29         }
30         else if (sender == "Peer") {
31             senderPriority = 3;
32         }
33         else if (sender == "ImportantPerson") {
34             senderPriority = 2;
35         }
36         else {
37             senderPriority = 1;
38         }
39
40         // Convert MM-DD-YYYY into YYYYMMDD for comparison
41         int month = stoi(date.substr(0, 2));
42         int day = stoi(date.substr(3, 2));
43         int year = stoi(date.substr(6, 4));
44
45         dateValue = year * 10000 + month * 100 + day;
46     }
47
48     string getSender() const {
49         return sender;
50     }
51
52     string getSubject() const {
53         return subject;
54     }
55
56     string getDate() const {
57         return date;
58     }
59
60     int getSenderPriority() const {
61         return senderPriority;
62     }
63
64     int getDateValue() const {
65         return dateValue;

```

```

66     }
67
68     // Returns true if this email has higher priority
69     bool higherPriority(const Email& other) const {
70         if (senderPriority != other.senderPriority) {
71             return senderPriority > other.senderPriority;
72         }
73
74         return dateValue > other.dateValue;
75     }
76
77     void display() const {
78         cout << "Next email:" << endl;
79         cout << "Sender: " << sender << endl;
80         cout << "Subject: " << subject << endl;
81         cout << "Date: " << date << endl;
82         cout << endl;
83     }
84 };
85
86 class MaxHeap {
87 private:
88     vector<Email> heap;
89
90     // Restore heap property upward
91     void heapifyUp(int index) {
92         while (index > 0) {
93             int parent = (index - 1) / 2;
94
95             if (heap[index].higherPriority(heap[parent])) {
96                 swap(heap[index], heap[parent]);
97                 index = parent;
98             }
99             else {
100                 break;
101             }
102         }
103     }
104
105     // Restore heap property downward
106     void heapifyDown(int index) {
107         int size = heap.size();
108
109         while (true) {
110             int left = 2 * index + 1;
111             int right = 2 * index + 2;
112             int largest = index;
113
114             if (left < size &&
115                 heap[left].higherPriority(heap[largest])) {
116                 largest = left;
117             }
118
119             if (right < size &&
120                 heap[right].higherPriority(heap[largest])) {
121                 largest = right;
122             }
123
124             if (largest != index) {
125                 swap(heap[index], heap[largest]);
126                 index = largest;
127             }
128             else {

```

```

129         break;
130     }
131 }
132 }
133
134 public:
135     // Insert an email into the heap
136     void insert(const Email& email) {
137         heap.push_back(email);
138         heapifyUp(heap.size() - 1);
139     }
140
141     // Display highest-priority email without removing it
142     void next() const {
143         if (heap.empty()) {
144             return;
145         }
146
147         heap[0].display();
148     }
149
150     // Remove highest-priority email
151     void read() {
152         if (heap.empty()) {
153             return;
154         }
155
156         if (heap.size() == 1) {
157             heap.pop_back();
158             return;
159         }
160
161         heap[0] = heap.back();
162         heap.pop_back();
163
164         heapifyDown(0);
165     }
166
167     // Return number of unread emails
168     int count() const {
169         return heap.size();
170     }
171
172     bool isEmpty() const {
173         return heap.empty();
174     }
175 };
176
177 class EmailSystem {
178 private:
179     MaxHeap emailQueue;
180
181 public:
182     void processCommand(const string& command) {
183         if (command == "NEXT") {
184             emailQueue.next();
185         }
186         else if (command == "READ") {
187             emailQueue.read();
188         }
189         else if (command == "COUNT") {

```

```

190         cout << "There are "
191         |<< emailQueue.count()
192         << " emails to read." << endl;
193     }
194     else if (command.substr(0, 6) == "EMAIL ") {
195         string data = command.substr(6);
196
197         stringstream ss(data);
198
199         string sender;
200         string subject;
201         string date;
202
203         // Read sender category
204         getline(ss, sender, ',');
205
206         // Read subject
207         getline(ss, subject, ',');
208
209         // Read date
210         getline(ss, date);
211
212         // Remove leading spaces after commas
213         if (!subject.empty() && subject[0] == ' ') {
214             subject.erase(0, 1);
215         }
216
217         if (!date.empty() && date[0] == ' ') {
218             date.erase(0, 1);
219         }
220
221         Email newEmail(sender, subject, date);
222         emailQueue.insert(newEmail);
223     }
224 }
225 };
226
227 int main() {
228     EmailSystem system;
229     string command;
230
231     // Process commands until EOF
232     while (getline(cin, command)) {
233         if (!command.empty()) {
234             system.processCommand(command);
235         }
236     }
237
238     return 0;
239 }

```

Claude:

```

1 // CEO Email Prioritizer - custom list-based MaxHeap priority queue (no <queue>, no std heap algorithms)
2 #include <iostream>
3 #include <sstream>
4 #include <string>
5 #include <vector>
6 #include <cstdlib>
7
8 // -----
9 // Email: one message plus the logic for comparing priority.
10 // -----
11 class Email {
12 private:
13     std::string sender_;
14     std::string subject_;
15     std::string date_; // MM-DD-YYYY (kept as given for output)
16     int senderRank_; // higher = more important
17     long dateKey_; // YYYYMMDD, larger = newer
18     long sequence_; // arrival order, used only as a last-resort tie-break
19
20     static int rankOf(const std::string& s) {
21         if (s == "Boss") return 5;
22         if (s == "Subordinate") return 4;
23         if (s == "Peer") return 3;
24         if (s == "ImportantPerson") return 2;
25         return 1; // OtherPerson / unknown
26     }
27
28     static long keyOf(const std::string& d) {
29         int mm = 0, dd = 0, yyyy = 0;
30         char a = 0, b = 0;
31         std::istringstream in(d);
32         in >> mm >> a >> dd >> b >> yyyy;
33         return static_cast<long>(yyyy) * 10000L + mm * 100L + dd;
34     }
35
36 public:
37     Email(): senderRank_(0), dateKey_(0), sequence_(0) {}
38
39     Email(const std::string& sender, const std::string& subject,
40           const std::string& date, long sequence)
41         : sender_(sender), subject_(subject), date_(date),
42           senderRank_(rankOf(sender)), dateKey_(keyOf(date)), sequence_(sequence) {}
43
44     const std::string& getSender() const { return sender_; }
45     const std::string& getSubject() const { return subject_; }
46     const std::string& getDate() const { return date_; }
47
48     // True if this email should be read before 'other'.
49     bool hasHigherPriorityThan(const Email& other) const {
50         if (senderRank_ != other.senderRank_) return senderRank_ > other.senderRank_;
51         if (dateKey_ != other.dateKey_) return dateKey_ > other.dateKey_;
52         return sequence_ < other.sequence_; // earlier arrival wins ties
53     }
54 };
55
56 // -----
57 // MaxHeap: priority queue of Emails, backed by a dynamic array (std::vector).
58 // -----
59 class MaxHeap {
60 private:
61     std::vector<Email> data_;
62
63     static std::size_t parent(std::size_t i) { return (i - 1) / 2; }
64     static std::size_t left(std::size_t i) { return 2 * i + 1; }
65     static std::size_t right(std::size_t i) { return 2 * i + 2; }

```

```

66
67 void swapAt(std::size_t i, std::size_t j) {
68     Email tmp = data_[i];
69     data_[i] = data_[j];
70     data_[j] = tmp;
71 }
72
73 void siftUp(std::size_t i) {
74     while (i > 0) {
75         std::size_t p = parent(i);
76         if (data_[i].hasHigherPriorityThan(data_[p])) {
77             swapAt(i, p);
78             i = p;
79         } else {
80             break;
81         }
82     }
83 }
84
85 void siftDown(std::size_t i) {
86     const std::size_t n = data_.size();
87     while (true) {
88         std::size_t l = left(i), r = right(i), best = i;
89         if (l < n && data_[l].hasHigherPriorityThan(data_[best])) best = l;
90         if (r < n && data_[r].hasHigherPriorityThan(data_[best])) best = r;
91         if (best == i) break;
92         swapAt(i, best);
93         i = best;
94     }
95 }
96
97 public:
98     bool isEmpty() const { return data_.empty(); }
99     std::size_t size() const { return data_.size(); }
100
101     void insert(const Email& e) {
102         data_.push_back(e);
103         siftUp(data_.size() - 1);
104     }
105
106     // Precondition: !isEmpty()
107     const Email& peek() const { return data_[0]; }
108
109     // Removes the top element. Returns false if the heap was empty.
110     bool removeTop() {
111         if (data_.empty()) return false;
112         data_[0] = data_.back();
113         data_.pop_back();
114         if (!data_.empty()) siftDown(0);
115         return true;
116     }
117 };
118
119 // -----
120 // InboxManager: parses commands and drives the heap.
121 // -----
122 class InboxManager {
123 private:
124     MaxHeap heap_;
125     long nextSequence_;
126
127     static std::string trim(const std::string& s) {
128         const char* ws = " \t\r\n";
129         std::size_t b = s.find_first_not_of(ws);

```

```

130     if (b == std::string::npos) return "";
131     std::size_t e = s.find_last_not_of(ws);
132     return s.substr(b, e - b + 1);
133 }
134
135 void handleEmail(const std::string& args) {
136     // Format: <sender>, <subject>, <date> (subject may itself contain commas)
137     std::size_t first = args.find(',');
138     std::size_t last = args.rfind(',');
139     if (first == std::string::npos || last == first) return; // malformed
140     std::string sender = trim(args.substr(0, first));
141     std::string subject = trim(args.substr(first + 1, last - first - 1));
142     std::string date = trim(args.substr(last + 1));
143     heap_.insert(Email(sender, subject, date, nextSequence_++));
144 }
145
146 void handleNext() const {
147     if (heap_.isEmpty()) return; // nothing to show, do not crash
148     const Email& e = heap_.peek();
149     std::cout << "Next email:\n"
150     |         << "Sender: " << e.getSender() << "\n"
151     |         << "Subject: " << e.getSubject() << "\n"
152     |         << "Date: " << e.getDate() << "\n";
153 }
154
155 void handleRead() { heap_.removeTop(); } // silent; safe when empty
156
157 void handleCount() const {
158     std::cout << "There are " << heap_.size() << " emails to read.\n";
159 }
160
161 public:
162     InboxManager() : nextSequence_(0) {}
163
164     void processLine(const std::string& rawLine) {
165         std::string line = trim(rawLine);
166         if (line.empty()) return;
167
168         std::size_t sp = line.find_first_of(" \t");
169         std::string cmd = (sp == std::string::npos) ? line : line.substr(0, sp);
170         std::string args = (sp == std::string::npos) ? "" : trim(line.substr(sp + 1));
171
172         if (cmd == "EMAIL") handleEmail(args);
173         else if (cmd == "NEXT") handleNext();
174         else if (cmd == "READ") handleRead();
175         else if (cmd == "COUNT") handleCount();
176         // unknown commands are ignored
177     }
178
179     void run(std::istream& in) {
180         std::string line;
181         while (std::getline(in, line)) processLine(line);
182     }
183 };
184
185 int main() {
186     InboxManager manager;
187     manager.run(std::cin);
188     return 0;
189 }

```

4. Code Comparison:

In terms of correctness, both the code given by ChatGPT and Claude execute correctly when standard test inputs are used, successfully implementing a MaxHeap priority queue in C++ to organize emails by sender category and date. However, in terms of edge cases, ChatGPT's code contains a structural issue inside its input parsing logic. Specifically, ChatGPT parses lines using a standard stringstream with a fixed comma delimiter (getline(ss, subject, ",")), which fails if an email subject line contains internal

commas. Additionally, ChatGPT manually strips single leading space characters rather than using a full string trimming method. Claude's program avoids this parsing issue by locating the first comma (after the sender) and the last comma (before the date), ensuring subjects with internal commas remain intact. Claude also includes a custom trimming helper function that cleans up trailing whitespace, carriage returns (`\r`), and newlines across operating system boundaries, while also tracking arrival sequence order as a deterministic tie breaker for identical priority ranks.

As for execution time, both programs execute efficiently and have $O(\log N)$ time complexity for EMAIL insertion and READ extraction, as both move nodes up or down a binary tree structure. Both programs also run in $O(1)$ time complexity for NEXT and COUNT commands since peeking at the top node or querying the array size requires constant lookup. Claude is slightly better in performance during the console output operations because it uses `\n` instead of `std::endl`, preventing unnecessary output buffer flushes on every line.

In terms of space complexity, both programs maintain an $O(N)$ space complexity, where N represents the number of unread emails stored in dynamic memory inside a `std::vector` list container. Claude requires slightly more space per email element to store a sequence ID for tie-breaking, but overall memory consumption across both implementations is basically identical.

In terms of maintainability, Claude's implementation is better structured and adheres to better standard C++ practices. Claude encapsulates helper methods as static functions, avoids using too many global namespace by using standard `std::` prefixing instead of using namespace `std`;, marks non-modifying member functions as `const`, and uses trailing underscores for private class variables. In contrast, ChatGPT relies on global namespace declarations, simple variable naming, and less defensive string parsing routines.

5. Baseline Selection and Improvements:

For the baseline selection, I chose the code given by Claude, since it is more maintainable and has a much better input parsing implementation that cleanly handles edge cases like commas in subject lines and cross-platform carriage returns (`\r\n`).

As for the improvements, in terms of correctness, Claude's core logic was already fully functional and handled edge cases well, but I refined the output formatting of the NEXT command to strictly match the expected whitespace line breaks. For date handling, the date parsing was already accurate, so I kept the normalized YYYYMMDD integer logic intact without unnecessary modifications. As for execution time and space complexity, both remained optimal at $O(\log N)$ time complexity for heap operations and $O(N)$ space complexity for dynamically stored email records, though I optimized memory usage slightly by passing string arguments by const reference and adding `reserve()` to pre-allocate vector memory. Furthermore, the main improvements were made in maintainability, where I added a full file prologue header and comprehensive in-line comments detailing block logic and source attributions while still preserving Claude's modular structure and clean class definitions.